

Six-Minute Final Video

Script, shot list and on-screen plan for **Dhaka Tesla Pool**: Jashim drives Bullet (3 seats), Nusrat and Rafiq share it, and Shirin contends for the last seat.

0:00–1:00 Problem, users, core idea	1:00–3:00 Engineering: architecture, backend, frontend, database, ride/pool lifecycle, one key decision, one trade-off (show the diagrams)	3:00–6:00 Product tour: passenger, driver, shared-Tesla pooling, fare and status, edge case, deployment
---	--	---

Read this first. The spoken lines below are a draft in plain words. Read them through twice and then **say them your way**. The rubric wants "your own words", and reading word-for-word is easy to hear. Target pace: about 140 words per minute. Each block is sized to fit its time slot at that pace.

1. Before you press record

1.1 Get the system running (about 5 minutes)

What	How
Backend (8 containers)	<code>cd C:\Users\User\Desktop\Tesla-lagbo-failsafe</code> <code>GATEWAY_PORT=18000 NOTIFICATION_PORT=18005</code> <code>RABBITMQ_PORT=25672 RABBITMQ_UI_PORT=35672 REDIS_PORT=16379</code> <code>docker compose -p tesla-lagbo up -d</code> Check: <code>docker compose -p tesla-lagbo ps</code> shows everything <i>healthy</i> .
Frontend	<code>cd C:\Users\User\Desktop\Tesla-lagbo\frontend</code> → <code>npm run dev</code> → open http://localhost:3000
Reset the demo story	In the app: Simulation → Pool forming → Load . Nusrat is in seat 1, Rafiq in seat 2, seat 3 is open.
Fresh backend data (optional)	<code>docker compose -p tesla-lagbo down -v</code> then <code>up -d --build</code> , so the e2e script shows 52/52 instead of 51/52. This deletes the demo databases.

1.2 Tabs and windows to have open, in this order

1. **README**, at the architecture diagram (mermaid flowchart) and the ERD (section 3 of this guide has both).
2. **VS Code** with three files ready:
 - `services/trip/app/state_machine.py` (transition table)
 - `services/trip/app/pooling.py`, around line 56 (the conditional seat UPDATE)
 - `services/trip/app/models.py`, around line 47 (`ck_pool_capacity` CHECK constraint)
3. **Browser**: `http://localhost:3000` (Passenger Console, viewing as Shirin), window about 1440 px wide.
4. **Terminal** in the failsafe folder, ready to run the e2e script (section 4, segment 3F).

5. (Optional) **RabbitMQ UI** <http://localhost:35672>, Queues tab.

Decide before recording: fare numbers. The frontend shows the guideline's fare (₹30 base + ₹45 distance – 30% = ₹52.50 for Rafiq). The backend fare service charges ₹54.00 for the same trip: ₹30 base + ₹30 distance – 20% of distance. Either:

- (a) align the frontend tariff with the backend before recording (a small change in `frontend/src/domain/fare.ts`), or
- (b) keep it, and use the "honesty line" given in segment 3D.

Don't show two different prices without saying why. Evaluators will notice.

Say it once, plainly: the web UI runs on an in-browser dispatch model that ports the backend's rules. It is not calling the gateway yet. The script covers this in segment 3A and proves the real backend with the end-to-end run in segment 3F.

1.3 Recording settings

- **Tool:** Loom (free, 5-minute cap on the free tier) or OBS / Windows Game Bar (Win+Alt+R, no length limit). If Loom's free cap is 5 minutes for you, use OBS: the brief allows up to 6.
- **Setup:** 1080p, full screen, mic close to you, notifications off, browser zoom 110–125% so text is readable.
- **Webcam bubble** on for the first minute (it builds trust); it can stay small in a corner after that.
- Do one full dry run with a timer. **Hard stop at 5:55.**

2. Minute-by-minute script

Left column: what's on screen and what you do. Right column: what you say.

Part 1: The problem, the users, the idea (0:00–1:00)

1A · Hook and problem		0:00–0:25
SHOW You on webcam, or the app's home screen (Passenger Console) as a still backdrop.	SAY "Hi, I'm [name], and this is Dhaka Tesla Pool. On a road like Banani to Mohakhali, a small electric Tesla carries one or two people when it could carry three. The seats are wasted, fares stay high, and riders wait longer. The problem isn't finding <i>a</i> car. It's filling the empty seats in cars that are already going the same way, safely and fairly."	
1B · Users		0:25–0:45
SHOW Point at the persona cards: Nusrat, Rafiq, Shirin . Then click the Driver Cockpit tab for a second to show Jashim.	SAY "There are two kinds of user. Passengers, like Nusrat, who needs to get to Mohakhali, and Rafiq, who's going to Gulshan 1 from the same street. And drivers, like Jashim, who drives a three-seat Tesla called Bullet. Passengers want certainty: am I booked, when is he here, what do I pay. Jashim wants one clear next step while he's driving in traffic."	
1C · Core idea		0:45–1:00
SHOW The Bullet cabin card: seats 1 and 2 taken, seat 3 open.	SAY "The core idea: riders leaving from the same area whose drop-offs fit on one route share one vehicle. Each pays for their own direct distance with a pool discount, never for someone else's detour. And the system must never sell the same seat twice, even when two people tap at the same moment."	

Part 2: How I engineered it (1:00–3:00)

2A · Architecture

1:00–1:35

SHOW

The **architecture diagram** (section 3, figure 1). Move your cursor along it: client → gateway → services → RabbitMQ / Redis.

SAY

"The backend is six Python FastAPI services, each owning one job. The **gateway** is the only door in: it checks the JWT, rate-limits, and handles idempotency keys, so a retried booking on a bad mobile connection is never booked twice. **Identity** knows who you are and what you drive. **Matching** knows the zones, the distances, and where drivers are. **Trip** owns rides and pools. **Fare** prices and settles. **Notification** pushes live updates over a WebSocket."

"The rule for how they talk: HTTP when I need an answer right now, like 'what does this cost'. RabbitMQ events when I'm announcing something happened, like 'ride completed'. And Redis pub/sub for high-frequency GPS pings."

2B · Database design

1:35–2:00

SHOW

The **ERD** (section 3, figure 2). Circle pools and ride_requests in the Trip database, then fares in the Fare database.

SAY

"Each service has its own database, so no service reaches into another's tables. The heart is the Trip database. A **pool** is one trip of one vehicle, with its capacity and occupied seats. **Ride requests** belong to a pool. **Waypoints** are the planned stop order, and **status history** keeps an audit trail of every transition. Money is always an integer number of poyscha: ninety-three taka fifty is stored as 9,350, so there are no rounding errors. Every service writes its events to an **outbox** table in the same transaction, which means an event can't be lost if the broker is down."

2C · Ride and pool lifecycle

2:00–2:25

SHOW

state_machine.py (the TRANSITIONS table), or the lifecycle diagram (section 3, figure 3).

SAY

"A ride moves through one explicit state machine: requested, matched, driver arrived, started, completed, with cancelled branching off the early states. Each transition says who's allowed to make it. A passenger can cancel while it's requested or matched. Once the driver has arrived, only the driver can cancel, as a no-show. Once started, nobody can. A pool goes from forming, to in progress when the first rider boards, to completed. After it starts, nobody new can join."

2D · One key decision

2:25–2:45

SHOW

pooling.py ~line 56: the UPDATE ...
WHERE occupied_seats + seats <=
max_capacity AND version =
Then the ck_pool_capacity CHECK
in models.py.

SAY

"My key decision was how to stop overbooking. I don't read the seat count and then write it back. Two requests could both read 'one seat left'. Instead, a single conditional UPDATE only succeeds if the seats still fit and the pool version hasn't changed, inside a write-locked transaction. A database CHECK constraint backs it up. If two riders race for the last seat, exactly one row updates. The other gets a clean 'seat taken' answer and is put back into search."

2E · One trade-off, and the frontend

2:45–3:00

SHOW

Back to the architecture diagram.
Then flash
frontend/src/design/tokens.ts
for 2 seconds.

SAY

"The trade-off: I used SQLite, one file per service. It's simple, and its single writer makes that seat check race-free. But each service can only run as one container, so it doesn't scale out. The path to scale is Postgres per service. The frontend is Next.js with TypeScript, Radix and Framer Motion, built on one design-token file, with no hard-coded colours or spacing, and WCAG AA contrast checked in tests."

Part 3: Product tour (3:00–6:00)

3A · Passenger flow

3:00–3:45

SHOW

1. Passenger Console, viewing as **Shirin**.
2. Open the **Pickup** dropdown (Banani, marked "Bullet pool"), then **Drop-off** → **Gulshan 2**.
3. Point to the green line "Shares Bullet's route... longest co-rider detour 135% (cap 140%)".
4. Point to the progress bar: Requested · Matched · In Transit · Completed.

SAY

"This is the passenger console. The UI runs against an in-browser model of the dispatch server that applies the same pooling rules as the backend, so it can be demoed without the stack; I'll show the real backend in a minute. I'm Shirin. I pick my pickup and drop-off from Dhaka's predefined zones. Banani is marked as where Bullet's pool is leaving from. The app tells me straight away whether I fit. Adding my Gulshan 2 stop keeps every co-rider's trip within 140% of their direct route, so I'm allowed in. At the top is my trip progress, and at the bottom, in thumb reach, is my one main action."

3B · Pooling: the shared Tesla

3:45–4:15

SHOW

1. The **Bullet cabin** card: seat 1 Nusrat "En Route to Mohakhali", seat 2 Rafiq "En Route to Gulshan 1", seat 3 open (pulsing border).
2. Switch persona to **Nusrat** for 3 seconds: her card says "Matched with Jashim" with the pickup ETA ticking.
3. Switch back to **Shirin**.

SAY

"Here's the shared Tesla. Nusrat booked first, and Jashim accepted her. Rafiq asked for the same corridor a bit later and joined automatically: the first rider needs the driver to accept, later compatible riders just join. Each seat shows where its rider is going, but not what they pay. Billing stays private. If I switch to Nusrat, she sees Jashim, Bullet's plate, and a live pickup countdown."

3C · Edge case: two people, one seat

4:15–4:50

SHOW

1. As Shirin, turn on "**Simulate Shirin Concurrent Booking**".
2. Click **Reserve Seat 3**. Pause to let it play: "Securing Seat..." → button shakes and turns red → seat 3 becomes **Nusrat** → red banner "Seat 3 was claimed by another commuter 340ms ago...".
3. Point to the route boxes: still Banani → Gulshan 2.

SAY

"Now the interesting edge case. There's one seat left, and Shirin and Nusrat tap for it at almost the same moment. Watch: 'securing seat', and then Nusrat's transaction lands 340 milliseconds earlier. Shirin doesn't get a crash or an error pop-up. The button shakes, the seat flips to Nusrat, and a banner explains what happened. It's also read out to screen readers. Her pickup and drop-off are kept, and she's automatically searching for the next pooled Tesla. If a seat frees up, she gets it without doing anything."

3D · Fare and status

4:50–5:10

SHOW

Switch to **Rafiq** → the Fare card shows the total → open "**View Verified Fare Breakdown**".

SAY

"Rafiq sees one clean number up front. One tap opens the verified breakdown: base rate, plus a distance charge, minus the pool discount, shown in taka and in poysha. Every figure is an integer in poysha under the hood, so it can be checked by hand."

If you did NOT align the fares, add: "This demo uses the guideline's tariff. The live fare service uses a 20% discount on the distance part only, which is why the API quotes ₳54.00 for the same trip. Unifying them is the next step."

3E · Driver flow

5:10–5:35

SHOW

1. First, **Simulation** → **Pool forming** → **Load** (resets the race).
2. **Driver Cockpit** tab: Online switch, battery, "2 / 3".
3. Click **Confirm Arrival** → drag the **Confirm Passenger Boarding** slider.
4. Point to the pipeline: Stop 1 ✓, next stop "Banani Star Kabab".

SAY

"This is Jashim's cockpit. Online status, battery, and how many seats are filled, then one big next stop. He confirms arrival, and boarding is a slide, not a tap, so a bump in traffic can't start a trip by accident. The stops run in order: pick up Nusrat, pick up Rafiq, drop Rafiq at Gulshan 1, drop Nusrat at Mohakhali. When he completes a drop-off, the fare settles into his ledger."

3F · The real backend and deployment

5:35–5:58

SHOW

Terminal (pre-run, just scroll the output):
docker compose -p tesla-lagbo
ps (all healthy)
COMPOSE_PROJECT_NAME=tesla-lagbo
E2E_API=http://localhost:18000/api/v1
E2E_WS=ws://localhost:18005/ws
E2E_RABBIT_UI=http://localhost:35672
..\Tesla-lagbo\venv\Scripts\python
scripts\e2e.py
End on the "checks passed" line.

SAY

"And the real backend: the whole stack runs with Docker Compose: six services, RabbitMQ and Redis. This end-to-end script plays the same Banani story through the real gateway and WebSocket, and checks every step, including that no event got stuck in a dead-letter queue. It's deployed locally with Compose; a cloud deployment is the next step. Thanks for watching!"

Timing buffer. The script comes to about 5:55. If you run long, cut these first:

- the Nusrat switch in 3B,
- the RabbitMQ sentence in 2A,
- the frontend sentence in 2E.

Never cut the edge case (3C). It's your strongest moment.

3. Diagrams to show (1:00–3:00)

Put these in the README, or screenshot this page. The rubric asks you to show your architecture and ERD while you explain.

Figure 1 · Architecture

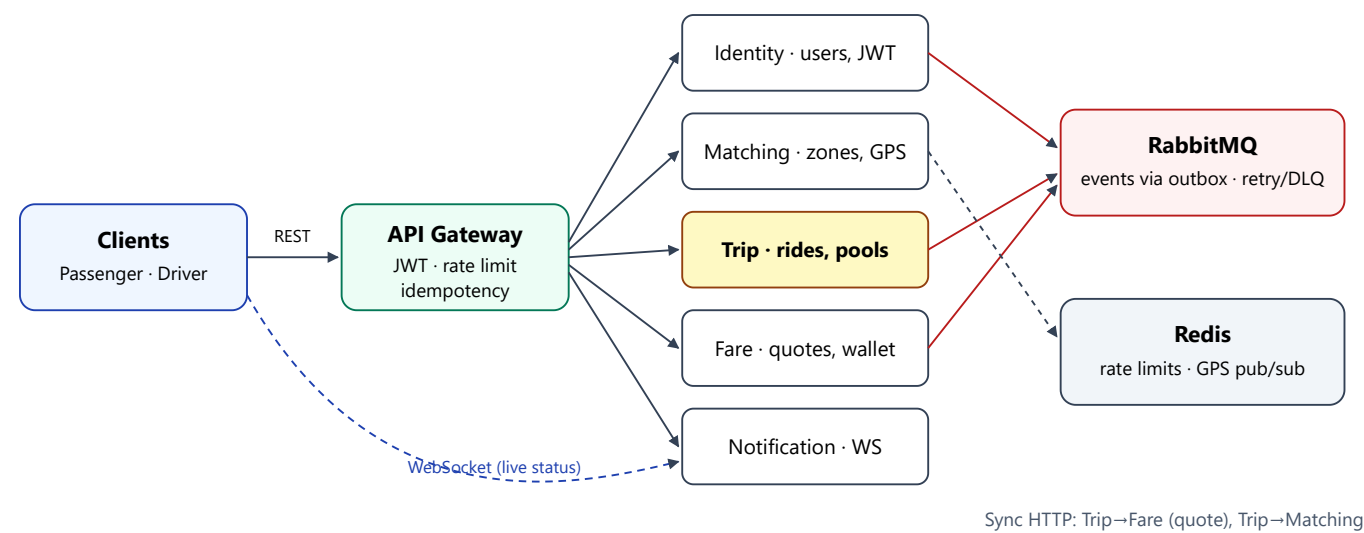


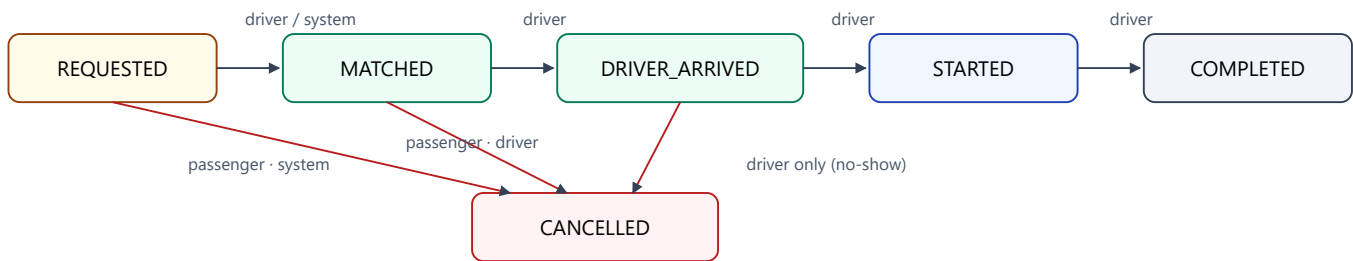
Figure 2 · ERD (one database per service)

Service DB	Tables and key columns	Relationships / constraints
identity.db	users(id, full_name, phone, password_hash, role) drivers(user_id, license_number, status) vehicles(id, driver_id, nickname, plate, seat_capacity) · outbox	users 1–1 drivers 1–1 vehicles; phone, license and plate are unique
matching.db	zones(code, name, lat, lng) zone_distances(from_zone, to_zone, distance_m)	9 Dhaka zones; live driver positions are in Redis, not SQL
trip.db	pools(id, driver_id, vehicle_id, max_capacity, occupied_seats, status, pickup_zone, version) ride_requests(id, passenger_id, pool_id, seats, pickup_zone, dropoff_zone, status, quote_id, estimated / final fare in poysha) pool_waypoints(pool_id, ride_request_id, seq, kind, zone, done_at) ride_status_history(ride_id, from_status, to_status, actor) ride_offers · driver_shifts · outbox · processed_events	pools 1–N ride_requests 1–N waypoints; history is an audit log. CHECK 0 ≤ occupied_seats ≤ max_capacity ; pickup ≠ dropoff; statuses are enumerated
fare.db	tariffs(base_poysha, per_km_poysha, pool_discount_pct) quotes(id, passenger_id, zones, distance_m, solo / pooled total, expires_at) fares(ride_id, base, distance_charge, pool_discount, total_poysha, payment_status) wallets(user_id, balance_poysha) · wallet_transactions · outbox · processed_events	quote → fare → wallet_transactions; all money is INTEGER poysha

notification.db	notifications(id, user_id, type, payload JSON, read_at) · processed_events	inbox for catch-up after reconnect
------------------------	---	---------------------------------------

Across services, links are by id only (for example, ride_requests.quote_id → fares.quote_id). There are no cross-database foreign keys. Consistency comes from events plus idempotent consumers (processed_events).

Figure 3 · Ride lifecycle (who can move it)



Pool: FORMING → IN_PROGRESS (first boarding; no new riders after this) → COMPLETED / CANCELLED. In the UI, STARTED is labelled "In Transit".

4. Cheat sheet: facts you might need

Topic	One-liner
Pooling rule	Same pickup zone, pool not started yet, seats fit, and no rider's in-vehicle distance goes above 140% of their direct distance.
Who confirms	The first rider needs the driver to accept (a 15-second offer). Later compatible riders join automatically.
Backend fare (fare service)	(base ₹30 + ₹15/km × km – 20% of the distance charge if pooled) × seats. Nusrat, Banani→Mohakhali 3.5 km: solo ₹82.50, pooled ₹72.00.
Frontend demo fare	(₹30 + ₹22.50/km × km) × (1 – 30%). Rafiq, 2.0 km: ₹52.50.
Money	INTEGER poysha everywhere (100 poysha = ₹1), always rounded down.
Reliability	Transactional outbox (at-least-once delivery) and processed_events (consumers can safely see the same event twice); retry queue, then dead-letter queue after 3 tries.
Double tap on "Book"	Gateway idempotency keys: the same key returns the saved response instead of booking again.
Privacy	Notifications send an allow-listed set of fields per recipient. Passengers never receive co-riders' fares.
Accessibility	48×48 px touch targets, WCAG AA contrast (tested), axe-core e2e, live regions, reduced motion respected.
Tests	Per-service pytest suites, frontend: 73 unit + 14 Playwright e2e, backend e2e script (Banani story).

5. Final checklist

- ☐ Stack healthy (`docker compose -p tesla-lagbo ps`), frontend on :3000, story reset to "Pool forming".
- ☐ Fare decision made (align, or use the honesty line).
- ☐ Architecture diagram and ERD visible in the README.
- ☐ e2e output ready to scroll in the terminal.
- ☐ One dry run, timed to under 5:55.
- ☐ Upload, set the link to "anyone with the link can view", and test it in a private window.
- ☐ Add the link at the **top** of the README:

📺 Demo video (6 min): [Watch on Loom](https://www.loom.com/share/...)